Lecture 4 : Dependency Injection, IoC & Loosely Coupled Design

1. Why Do We Need Spring Core?

Java already gives us many powerful building blocks:

- Classes
- Objects
- Constructors
- Interfaces
- Inheritance
- Packages

So a natural question appears:

> If Java already gives us all these features, why do we need Spring Core?

Spring Core is not mainly about building web applications. That part comes later with Spring MVC and Spring Boot.

The core idea of Spring is simpler:

> Spring helps us create objects, manage dependencies, and connect objects together in a clean and scalable way.

In small programs, object creation looks easy. But in real applications, one class depends on another class, which depends on another class, and this chain keeps growing.

That is where Spring Core becomes useful.

---

2. Understanding Dependency

Suppose we have an `EmailService` class:

```
class EmailService {

    public void sendEmail() {
        System.out.println("Email sent");
    }
}
```

Now suppose `OrderService` needs to send an email after placing an order.

```
class OrderService {

    private EmailService emailService = new EmailService();

    public void placeOrder() {
        System.out.println("Order placed");
        emailService.sendEmail();
    }
}
```

Here, `OrderService` depends on `EmailService`.

In simple words:

> A dependency is something a class needs in order to complete its work.

So in this example:

```
private EmailService emailService = new EmailService();
```

means:

> `OrderService` is creating its own dependency.

This works, but it creates a design problem.

3. The Problem: Tight Coupling

There are two broad design styles:

```
Tightly Coupled Design  → Hard to change
Loosely Coupled Design  → Easier to change
```

Real-Life Example

Imagine a person wants to travel from Delhi to Chandigarh.

Tightly Coupled Thinking

> "I can travel only by this one specific car, with this one specific driver, from this one specific company."

If that car is unavailable, the person is stuck.

Loosely Coupled Thinking

> "I need transportation. It can be a car, bus, train, or cab."

Here, the person is not dependent on one fixed option. The person only cares about the service: transportation.

---

4. Tight Coupling in Code

In our previous example, `OrderService` directly creates `EmailService`.

```java
class OrderService {

    private EmailService emailService = new EmailService();

    public void placeOrder() {
        System.out.println("Order placed");
```

```
        emailService.sendEmail();
    }
}
```

This means:

```
OrderService
    |
    | directly creates
    v
EmailService
```

Now suppose the requirement changes.

Instead of email, we now need to send an SMS.

```
public class SmsService {

    public void sendSms() {
        System.out.println("SMS notification sent to custome
r");
    }
}
```

Now we must modify `OrderService`.

```
public class OrderService {

    private SmsService smsService = new SmsService();

    public void placeOrder() {
        System.out.println("Order placed successfully");
        smsService.sendSms();
    }
}
```

The problem is not that the notification requirement changed.

The real problem is:

> A change in notification logic forced us to modify `OrderService`.

That means `OrderService` is tightly coupled to a specific notification class.

---

5. What Exactly Is Tight Coupling?

Tight coupling means:

> One class is directly dependent on a specific concrete class.

In our case:

```
OrderService is directly dependent on EmailService.
```

This makes the code harder to change, test, and reuse.

---

6. First Improvement: Use an Interface

To make the design better, we can introduce an interface.

```java
public interface NotificationService {

    void sendNotification();
}
```

Now different notification services can implement this interface.

```java
public class EmailService implements NotificationService {

    @Override
    public void sendNotification() {
        System.out.println("Email sent");
```

```
    }
}
```

```
public class SmsService implements NotificationService {

    @Override
    public void sendNotification() {
        System.out.println("SMS sent");
    }
}
```

Now `OrderService` can depend on the interface instead of a concrete class.

```
public class OrderService {

    private NotificationService notificationService = new Ema
ilService();

    public void placeOrder() {
        System.out.println("Order placed successfully");
        notificationService.sendNotification();
    }
}
```

This is better than before because the variable type is now an interface:

```
NotificationService
```

But there is still one issue.

---

7. Interface Alone Is Not Enough

Even though we are using an interface, this line is still a problem:

```
private NotificationService notificationService = new EmailSe
rvice();
```

Why?

Because the object creation is still concrete:

```
new EmailService()
```

So `OrderService` is still deciding which implementation to use.

The important point is:

> Creating objects is not the problem.
>
> Creating them in the wrong place is the problem.

`OrderService` is a business class. Its job should be order-related logic, not deciding which notification object to create.

Currently, `OrderService` is doing two things:

1. Handling order-related logic
2. Creating and deciding the notification service

This breaks two important design principles:

```
S → Single Responsibility Principle
O → Open-Close Principle
```

The main idea is:

> Dependency is normal. Tight dependency is the problem.

---

8. The Solution: Dependency Injection

Instead of creating the dependency inside `OrderService`, we provide the dependency from outside.

```
public class OrderService {

    private NotificationService notificationService;

    public OrderService(NotificationService notificationServi
ce) {
        this.notificationService = notificationService;
    }

    public void placeOrder() {
        System.out.println("Order placed successfully");
        notificationService.sendNotification();
    }
}
```

Now `OrderService` does not create `EmailService` or `SmsService`.

It simply says:

> "Give me something that can send a notification."

That "something" is represented by the `NotificationService` interface.

---

9. Object Creation Happens Outside

Now we create the objects in `Main`.

```
public class Main {

    public static void main(String[] args) {

        NotificationService notificationService = new EmailSe
rvice();

        OrderService orderService = new OrderService(notifica
tionService);
```

```
        orderService.placeOrder();
    }
}
```

Now the dependency is provided from outside.

This means:

```
OrderService depends only on NotificationService.
OrderService does not create EmailService.
OrderService receives the dependency from outside.
```

This is called **Dependency Injection**.

---

10. What Is Dependency Injection?

Dependency Injection means:

> A class receives the objects it depends on from outside, instead of creating those objects itself.

A simpler way to remember it:

> Do not create your dependency. Receive your dependency.

Or:

> A class should ask for what it needs, not build everything by itself.

Dependency Injection is not only a Spring concept.

It is a design principle that we can use in plain Java also.

Spring did not invent Dependency Injection. Spring automates it.

---

11. Benefits of Dependency Injection

Benefit 1: Easy to Change Implementation

If we want to switch from email to SMS, we do not need to change `OrderService`.

We only change the object passed from outside.

```
NotificationService notificationService = new SmsService();
OrderService orderService = new OrderService(notificationServ
ice);
```

`OrderService` remains unchanged.

Benefit 2: Easier to Test

For testing, we may not want to send a real email or SMS.

So we can create a fake implementation.

```
public class FakeNotificationService implements NotificationS
ervice {

    @Override
    public void sendNotification() {
        System.out.println("Fake notification for testing");
    }
}
```

Now we can test `OrderService` without depending on real email or SMS logic.

Benefit 3: More Reusable Code

The same `OrderService` can work with multiple implementations:

```
EmailService
SmsService
WhatsAppService
FakeNotificationService
```

This makes the design flexible and reusable.

12. Types of Dependency Injection

There are mainly three common types of Dependency Injection.

1. Constructor Injection

In constructor injection, the dependency is provided through the constructor.

```
public class OrderService {

    private NotificationService notificationService;

    public OrderService(NotificationService notificationServi
ce) {
        this.notificationService = notificationService;
    }
}
```

Constructor injection is usually preferred because it makes required dependencies clear.

If `OrderService` cannot work without `NotificationService`, constructor injection is a good choice.

2. Setter Injection

In setter injection, the dependency is provided through a setter method.

```
public class OrderService {

    private NotificationService notificationService;

    public void setNotificationService(NotificationService no
tificationService) {
```

```
        this.notificationService = notificationService;
    }

    public void placeOrder() {
        System.out.println("Order placed successfully");
        notificationService.sendNotification();
    }
}
```

Setter injection is useful when the dependency is optional or can be changed later.

---

3. Field Injection

In Spring, we will also discuss about field injection later.

---

13. What Is IoC?

IoC stands for **Inversion of Control**.

To understand IoC, first ask:

> In the earlier code, who was controlling the creation of `EmailService`?

The answer is:

```
OrderService was controlling object creation.
```

So the control was inside `OrderService`.

After Dependency Injection, `OrderService` no longer creates `EmailService`.

The object is created outside and passed to `OrderService`.

This movement of control is called **Inversion of Control**.

---

14. Simple Definition of IoC

Earlier:

> The class created what it needed.

Now:

> The class receives what it needs.

That reversal is called **Inversion of Control**.

It is called “inversion” because the normal control flow is reversed.

Earlier:

```
OrderService
    |
    | creates
    v
EmailService
```

After Dependency Injection:

```
Main
 |
 | creates
 v
EmailService

Main provides EmailService
 |
 | to
 v
OrderService
```

So the control moved from inside the class to outside the class.

---

15. Relationship Between IoC and DI

IoC and DI are closely related, but they are not exactly the same.

```
IoC → Principle
DI  → Technique to achieve that principle
```

In simple words:

> IoC is the idea.
>
> Dependency Injection is one way to implement that idea.

When we give dependencies from outside instead of creating them inside the class, we are using Dependency Injection to achieve Inversion of Control.

---

16. Where Does Spring Fit In?

In plain Java, we moved object creation outside `OrderService`.

But now `Main` is doing all the object creation and wiring.

```
NotificationService notificationService = new EmailService();
OrderService orderService = new OrderService(notificationServ
ice);
```

This is fine for small applications.

But in large applications, there may be hundreds of classes and dependencies.

If `Main` creates and connects everything manually, it becomes messy.

So we need something that can:

```
Create objects
Manage objects
Connect objects together
```

That is where Spring comes in.

---

17. Spring IoC Container

Spring provides an external system called the **Spring IoC Container**.

The Spring IoC Container is responsible for:

```
Creating objects
Managing objects
Injecting dependencies
Connecting classes together
```

In plain Java, `Main` was acting like a small container.

In Spring, the Spring IoC Container does this work automatically.

---

18. What Does Spring Core Do?

At a high level, Spring Core does three major things:

```
1. Creates objects
2. Manages objects
3. Connects objects together
```

This is the foundation of Spring.

Before learning Spring MVC, Spring Boot, Spring Data JPA, or Spring Security, it is important to understand this core idea.

---

19. What Is a Bean?

In normal Java, we call them objects.

In Spring, objects managed by Spring are called **beans**.

Important point:

> Every Spring bean is an object, but every object is not necessarily a Spring bean.

Example:

```
EmailService emailService = new EmailService();
```

This is a normal Java object.

But if Spring creates and manages `EmailService`, then it becomes a Spring bean.

---

20. Manual Java Flow vs Spring Flow

Plain Java Flow

```
Main
 |
 | creates
 v
EmailService

Main
 |
 | creates
 v
OrderService

Main injects EmailService into OrderService
```

In this flow, `Main` is responsible for creating and connecting objects.

Spring Flow

```
Spring IoC Container
 |
 | creates
 v
EmailService Bean

Spring IoC Container
 |
 | creates and injects dependency
 v
OrderService Bean
```

In this flow, Spring creates the objects and connects them automatically.

21. Final Summary

Dependency Injection helps us design classes that do not create their own dependencies.

Instead, dependencies are provided from outside.

This makes the code:

```
Easier to change
Easier to test
Easier to reuse
More loosely coupled
```

IoC means the control of object creation moves from the class itself to an external system.

DI is one way to achieve IoC.

Spring takes this idea and automates it using the Spring IoC Container.

At the core level, Spring is mainly helping us with object creation, object management, and object wiring.

That is the foundation of Spring Core.